Trace the algorithm.
Choose the next state.

90 original multiple-choice questions

30 topics / 1 easy + 1 medium + 1 hard per topic

Sorting, heaps, graph concepts, BFS/DFS, DAGs, shortest paths, minimum spanning trees and union-find.

How to use this draft

Choose one answer (A-D) per question. Every question has exactly one best answer under its stated rules.

Try the questions first. The answer section begins on page 33 and includes short worked explanations and source references.

Easy tests one local step or definition. Medium follows an intermediate state. Hard combines several steps, an invariant, or a subtle boundary case.

Questions use original small arrays and graphs. They are practice questions tailored to the requested MCQ format, not claimed past-paper questions.

This version is for content review before any video production. No videos were created.

Source basis

The seven supplied lecture decks for Weeks 1-7, checked alongside the existing local timestamped automated transcripts of all 20 matching recordings. This task used transcripts, not a fresh manual listen. Technical ASR errors are resolved against the slides and explicit traces. See pages 48-49 for source details and convention corrections.

Coverage map

Each row contains three questions in Easy / Medium / Hard order.

01	Sorting records and stability	Q01-03	p.3
02	Insertion sort	Q04-06	p.4
03	Merge sort and stable merging	Q07-09	p.5
04	Quicksort: one-scan partition	Q10-12	p.6
05	Partition alternatives and pivot choice	Q13-15	p.7
06	Counting sort	Q16-18	p.8
07	LSD radix sort	Q19-21	p.9
08	Sorting cost, space and lower bounds	Q22-24	p.10
09	Heap shape, order and indexing	Q25-27	p.11
10	Min-Heapify: repair downward	Q28-30	p.12
11	Bottom-up Build-Min-Heap	Q31-33	p.13
12	Heap-based priority queue	Q34-36	p.14
13	Heapsort and the active boundary	Q37-39	p.15
14	Graph representations and degrees	Q40-42	p.16
15	Paths, induced subgraphs and components	Q43-45	p.17
16	Breadth-first search (BFS)	Q46-48	p.18
17	Recursive depth-first search (DFS)	Q49-51	p.19
18	DFS tree, forward, back and cross edges	Q52-54	p.20
19	DFS cycle detection and full forests	Q55-57	p.21
20	DAGs and source-removal topological sort	Q58-60	p.22
21	DFS topological sort and partial orders	Q61-63	p.23
22	Shortest paths, relaxation and negative cycles	Q64-66	p.24
23	Bellman-Ford passes and detection	Q67-69	p.25
24	Dijkstra: labels, parents and settling	Q70-72	p.26
25	DAG shortest paths and algorithm selection	Q73-75	p.27
26	MST objective, cuts and cycle property	Q76-78	p.28
27	Prim: cheapest attachment edges	Q79-81	p.29
28	Union-find: naive linking and compression	Q82-84	p.30
29	Union by rank with path compression	Q85-87	p.31
30	Kruskal: accepted edges and forest state	Q88-90	p.32

Default rules: arrays are zero-based; numerical sorts are ascending. Every changed heap convention is stated. Graph arrows indicate direction; line crossings are not vertices.

A question restarts from its section input unless it explicitly says to continue. Arrays and vectors show full states unless an active heap boundary | is drawn.

INF means unreachable. Queue order is front-to-back; DFS stack order is bottom-to-top. Strict relaxation means equal candidates do not change parents.

Topic 25 is labelled as a recording-mentioned tutorial extension. Later slide decks (analysis, greedy, DP, strings, NP) are excluded from this Weeks 1-7 bank.

01 Sorting records and stability

WEEK 1

A record such as 2a has key 2 and identity a. Sort ascending by key; identities move with their keys.

Q01 / EASY

Stably sort [3a, 1x, 3b, 2y]. Which final array is correct?

A. [1x, 3a, 2y, 3b]
B. [1x, 2y, 3b, 3a]
C. [1a, 2x, 3b, 3y]
D. [1x, 2y, 3a, 3b]

Q02 / MEDIUM

Records are [(B,2), (A,3), (B,1), (A,1)]. To sort by letter first and number second, which sequence of stable sorts gives the required final order?

A. Number first, then letter; [(A,3), (A,1), (B,2), (B,1)]
B. Letter first, then number; [(A,3), (A,1), (B,2), (B,1)]
C. Number first, then letter; [(A,1), (A,3), (B,1), (B,2)]
D. Letter first, then number; [(A,1), (B,1), (B,2), (A,3)]

Q03 / HARD

A black-box sorter maps [2a, 1x, 2b, 1y] to [1x, 1y, 2b, 2a]. What can this single observation establish?

A. It produced a sorted permutation, but the sorter is not stable.
B. It failed to preserve the input records.
C. It is stable because the keys are sorted.
D. It uses O(1) auxiliary words.

TRACE / WORKING

02 Insertion sort

WEEK 1

Use the slide version: for i=1..n-1, swap adjacent entries while A[j] > A[j+1]. Indices are zero-based.

Q04 / EASY

For [7, 3, 5, 2], what is the array after the outer iteration i=1 finishes?

A. [7, 3, 2, 5]
B. [3, 7, 5, 2]
C. [3, 5, 7, 2]
D. [2, 3, 5, 7]

Q05 / MEDIUM

For [7, 3, 5, 2], what is the array after i=2 finishes?

A. [3, 7, 2, 5]
B. [2, 3, 5, 7]
C. [3, 5, 7, 2]
D. [3, 5, 2, 7]

Q06 / HARD

For [4, 2a, 2b, 1], how many adjacent swaps occur over the entire sort, and what is the final record order?

A. 4 swaps; [1, 2a, 2b, 4]
B. 5 swaps; [1, 2a, 2b, 4]
C. 5 swaps; [1, 2b, 2a, 4]
D. 6 swaps; [1, 2b, 2a, 4]

TRACE / WORKING

03 Merge sort and stable merging

WEEK 2

Use top-down MergeSort on half-open ranges [start,end), mid=floor((start+end)/2). Recurse left first; on equal keys take the left record.

Q07 / EASY

For [8, 3, 6, 1], what is the whole array immediately after MergeSort finishes the left half, before it starts the right half?

A. [3, 8, 6, 1]

B. [8, 3, 1, 6]

C. [1, 3, 6, 8]

D. [3, 8, 1, 6]

Q08 / MEDIUM

Merge left [1a, 4a, 4b] with right [1b, 3, 4c]. What are the first four output records?

A. [1a, 1b, 3, 4a]

B. [1a, 3, 1b, 4a]

C. [1b, 1a, 3, 4c]

D. [1a, 1b, 4a, 3]

Q09 / HARD

Sort [5, 1, 4, 2, 3]. Immediately before the top-level merge, what are the two sorted halves, and how many front-key comparisons does that merge make?

A. [1,5] and [2,3,4]; 3 comparisons

B. [1,5] and [2,3,4]; 5 comparisons

C. [1,5] and [2,3,4]; 4 comparisons

D. [1,4,5] and [2,3]; 4 comparisons

TRACE / WORKING

04 Quicksort: one-scan partition

WEEK 2

PARTITION on inclusive [lo,hi]: p=A[hi], j=lo; scan i=lo..hi-1; if A[i]<p swap A[i],A[j] then j++; finally swap A[j],A[hi] and return j. Recurse [lo,j-1] then [j+1,hi].

Q10 / EASY

Partition [6, 2, 5, 1, 4] once. What array and pivot index are returned?

A. [2, 1, 4, 6, 5]; j=2
B. [2, 1, 5, 6, 4]; j=4
C. [1, 2, 4, 5, 6]; j=2
D. [2, 4, 5, 1, 6]; j=1

Q11 / MEDIUM

In that same partition, immediately after processing scan index i=2 (before i=3), what are A and j?

A. [2, 6, 5, 1, 4]; j=1
B. [6, 2, 5, 1, 4]; j=0
C. [2, 6, 4, 1, 5]; j=2
D. [2, 1, 5, 6, 4]; j=2

Q12 / HARD

Use [4a, 2, 4b, 1, 4c], comparing keys only. After the first full partition, which result and next recursive ranges are correct?

A. [2, 1, 4c, 4a, 4b]; [0,1] and [3,4]
B. [2, 1, 4c, 4b, 4a]; [0,1] and [3,4]
C. [2, 1, 4a, 4b, 4c]; [0,1] and [3,4]
D. [2, 1, 4c, 4a, 4b]; [0,2] and [2,4]

TRACE / WORKING

05 Partition alternatives and pivot choice

WEEK 2

Two-sided scan below uses a fixed pivot value p=4, not a distinguished pivot position. Scan left to the first value >=4 and right to the first value <=4; swap if the pointers have not crossed.

Q13 / EASY

On [1, 7, 3, 6, 2], what is the array after the first two-sided swap?

A. [1, 2, 3, 6, 7]

B. [1, 3, 7, 6, 2]

C. [1, 2, 3, 7, 6]

D. [2, 7, 3, 6, 1]

Q14 / MEDIUM

Three-way partition [4a, 1, 4b, 6, 2, 4c] around key 4. Which describes the three blocks and the elements still needing recursive sorting?

A. < block contains {1}; = block {2,4a,4b,4c}; > block {6}; recurse on < and >.

B. < block contains {1,2}; = block {4a,4b,4c}; > block {6}; recurse only on < and >.

C. < block contains {1,2,4a}; = block {4b}; > block {4c,6}; recurse on all blocks.

D. < block contains {1,2}; = block {4a,4b,4c}; > block {6}; recurse only on =.

Q15 / HARD

Run one-scan quicksort on already ascending distinct keys, always using the last key as pivot. Which describes the nontrivial partition sizes and total pivot comparisons?

A. (n-1,0), then (n-2,0), ...; n(n-1)/2 comparisons

B. Approximately equal halves each time; n-1 comparisons total

C. (n-1,0), then (n-2,0), ...; O(n log n) comparisons

D. (0,0) each time; n comparisons total

TRACE / WORKING

06 Counting sort

WEEK 2

Keys are 0..3. Use cumulative COUNTS C[v]=number of input keys <=v; scan input right-to-left, decrement C[key], then place at B[C[key]]. B is zero-based; _ means unfilled.

Q16 / EASY

For [2a, 0, 2b, 1, 3, 1b], what is the raw frequency array C before prefix sums?

A. [1, 2, 2, 1]
B. [1, 1, 2, 2]
C. [0, 1, 2, 3]
D. [1, 3, 5, 6]

Q17 / MEDIUM

For that input, after prefix sums but before any placements, which cumulative counts and first placement are correct?

A. C=[1,3,5,6]; first put 1b at B[3]
B. C=[0,2,4,5]; first put 1b at B[2]
C. C=[1,2,2,1]; first put 1b at B[1]
D. C=[1,3,5,6]; first put 1b at B[2]

Q18 / HARD

For that input, what are B and C after exactly three backward placements?

A. B=[_,_,1b,1,_,3]; C=[1,2,5,5]
B. B=[0,1,1b,2a,2b,3]; C=[0,1,3,5]
C. B=[_,1b,1,_,_,3]; C=[1,1,5,5]
D. B=[_,1,1b,_,_,3]; C=[1,1,5,5]

TRACE / WORKING

07 LSD radix sort

WEEK 2

Use base 10 LSD radix sort with stable digit passes, starting with units. Input is [329, 457, 657, 839, 436, 720, 355].

Q19 / EASY

What is the array after the units-digit pass?

A. [329, 720, 436, 839, 355, 457, 657]
B. [720, 355, 436, 457, 657, 329, 839]
C. [329, 355, 436, 457, 657, 720, 839]
D. [720, 355, 436, 657, 457, 839, 329]

Q20 / MEDIUM

What is the array after the tens-digit pass (two stable passes total)?

A. [329, 355, 436, 457, 657, 720, 839]
B. [720, 355, 436, 457, 657, 329, 839]
C. [720, 329, 436, 839, 355, 457, 657]
D. [720, 329, 839, 436, 355, 657, 457]

Q21 / HARD

After a correct units pass on [12, 11, 22, 21], the array is [11, 21, 12, 22]. An unstable tens pass reverses the order within each equal-tens group. What happens?

A. It produces [11,12,21,22], so stability was unnecessary.
B. It produces [21,22,11,12], which is numerically sorted.
C. It produces [11,21,12,22], which is numerically sorted.
D. It produces [12,11,22,21], so the two-pass result is not numerically sorted.

TRACE / WORKING

08 Sorting cost, space and lower bounds

WEEK 1-3

Auxiliary space includes recursive call stacks. In this course, in-place means O(1) additional words. Use the standard array-based implementations described in the questions.

Q22 / EASY

Which method sorts an arbitrary array in O(n log n) worst-case time using O(1) auxiliary words?

A. Merge sort using an auxiliary array
B. Counting sort with an output array and key counts
C. Heapsort using iterative heap repair
D. Quicksort using a recursive call stack

Q23 / MEDIUM

Counting sort receives n keys in 0..n^2-1 and allocates a count entry for every possible key. What is its time and auxiliary-space order?

A. Time Theta(n^2), space Theta(1).
B. Both are Theta(n^2).
C. Time Theta(n), space Theta(n).
D. Time Theta(n log n), space Theta(n).

Q24 / HARD

For n>=2, there are n word-sized keys in 0..n^3-1. Base-n radix sort uses stable counting sort for every digit. Which conclusion is correct?

A. At most 3 passes; O(n) time. This does not contradict the comparison-sort lower bound.
B. n passes; O(n^2) time because each key has n digits.
C. At most 3 passes; this disproves the Omega(n log n) comparison-sort lower bound.
D. 3 passes; O(n log n) time because every digit pass must use comparisons.

TRACE / WORKING

09 Heap shape, order and indexing

WEEK 3

Heap arrays are zero-based, level-order representations of complete binary trees. A min-heap requires every parent key <= each child key.

Q25 / EASY

For a heap with 10 entries, what are the parent and existing children of index 3?

- **A.** Parent 1; children 7 and 8
- **B.** Parent 6; children 7 and 8
- **C.** Parent 2; children 7 and 8
- **D.** Parent 1; children 6 and 7

Q26 / MEDIUM

Which array is a valid min-heap even though it is not fully sorted?

- **A.** [1, 4, 2, 3, 5, 8]
- **B.** [2, 1, 4, 8, 5, 3]
- **C.** [1, 4, 2, 8, 3, 5]
- **D.** [1, 4, 2, 8, 5, 3]

Q27 / HARD

A min-heap has 11 entries. Which statement gives its leaf indices and what can be inferred about its largest key?

- **A.** Leaves are 4..10; the largest key must be at index 4.
- **B.** Leaves are 5..10; at least one leaf contains a largest key, but its exact index is not fixed.
- **C.** Leaves are 6..10; the largest key must be at index 10.
- **D.** Leaves are 5..10; all leaves are larger than all internal nodes.

TRACE / WORKING

10 Min-Heapify: repair downward

WEEK 3

Min-Heapify compares both existing children, swaps with the smaller child only when it is smaller than the current key, then continues there. Child subtrees must already be min-heaps.

Q28 / EASY

For [9, 3, 5, 7, 8, 6], what is the array after the first swap in Min-Heapify(A,0)?

A. [5, 3, 9, 7, 8, 6]
B. [3, 9, 5, 7, 8, 6]
C. [6, 3, 5, 7, 8, 9]
D. [3, 7, 5, 9, 8, 6]

Q29 / MEDIUM

For the same input, what is the array after Min-Heapify(A,0) finishes?

A. [3, 9, 5, 7, 8, 6]
B. [3, 7, 5, 9, 8, 6]
C. [3, 8, 5, 7, 9, 6]
D. [3, 5, 6, 7, 8, 9]

Q30 / HARD

Call Min-Heapify(A,0) on [1, 9, 2, 3, 4, 5, 6]. Which result and guarantee are correct?

A. The array stays unchanged and is a min-heap.
B. The result is [1,3,2,9,4,5,6], a min-heap.
C. The result is fully sorted because the root is minimum.
D. The array stays unchanged; the whole array is still not a min-heap.

TRACE / WORKING

11 Bottom-up Build-Min-Heap

WEEK 3

Input is [9, 4, 7, 1, 3, 6, 2]. Build-Min-Heap calls Min-Heapify for i=floor(n/2)-1 down to 0; compare smaller children first.

Q31 / EASY

Which index is processed first, and what is the array after that complete heapify call?

A. [9, 4, 7, 1, 3, 6, 2]; first index 6
B. [1, 4, 7, 9, 3, 6, 2]; first index 0
C. [9, 1, 7, 4, 3, 6, 2]; first index 1
D. [9, 4, 2, 1, 3, 6, 7]; first index 2

Q32 / MEDIUM

What is the array after the next heapify call, at i=1, finishes?

A. [9, 1, 7, 4, 3, 6, 2]
B. [9, 3, 2, 1, 4, 6, 7]
C. [1, 3, 2, 4, 9, 6, 7]
D. [9, 1, 2, 4, 3, 6, 7]

Q33 / HARD

Which final heap and construction-time claim are correct?

A. [1, 3, 2, 4, 9, 6, 7]; Theta(n) construction time
B. [1, 4, 2, 9, 3, 6, 7]; Theta(log n) construction time
C. [1, 2, 3, 4, 6, 7, 9]; Theta(n) construction time
D. [1, 3, 2, 4, 9, 6, 7]; Theta(n log n) construction time

TRACE / WORKING

12 Heap-based priority queue

WEEK 3

Each question independently starts from min-heap [2, 5, 3, 9, 7, 8]. Insert appends and bubbles up; Extract-Min replaces the root by the last entry, shrinks, then repairs downward.

Q34 / EASY

Insert key 1. What is the final heap array?

A. [1, 2, 5, 3, 9, 7, 8]
B. [1, 2, 3, 5, 7, 8, 9]
C. [1, 5, 2, 9, 7, 8, 3]
D. [2, 5, 1, 9, 7, 8, 3]

Q35 / MEDIUM

Extract-Min once. Which returned key and remaining heap are correct?

A. Return 2; [3, 5, 7, 9, 8]
B. Return 2; [5, 7, 3, 9, 8]
C. Return 8; [2, 5, 3, 9, 7]
D. Return 2; [3, 5, 8, 9, 7]

Q36 / HARD

Insert 4, then Extract-Min twice. What keys are returned, and what heap remains?

A. Returns 4, then 2; [3, 5, 8, 9, 7]
B. Returns 2, then 4; [3, 5, 8, 9, 7]
C. Returns 2, then 3; [4, 5, 8, 9, 7]
D. Returns 2, then 3; [4, 5, 7, 9, 8]

TRACE / WORKING

13 Heapsort and the active boundary

WEEK 3

Use ascending heapsort with a max-heap. Start each question from max-heap [9, 7, 8, 1, 3, 2]. Each extraction swaps root with the active last entry, shrinks the heap and Max-Heapifies. | separates the sorted suffix.

Q37 / EASY

What is the array immediately after the first root/end swap, before shrinking and repair?

A. [9, 7, 8, 1, 3, 2]
B. [2, 7, 8, 1, 3, 9]
C. [1, 2, 3, 7, 8, 9]
D. [8, 7, 2, 1, 3, 9]

Q38 / MEDIUM

What is the state after the first extraction and its repair finish?

A. [9, 7, 8, 1, 3 | 2]
B. [8, 7, 2, 1, 3 | 9]
C. [8, 7, 3, 1, 2 | 9]
D. [7, 3, 8, 1, 2 | 9]

Q39 / HARD

What is the state after two complete extractions and repairs?

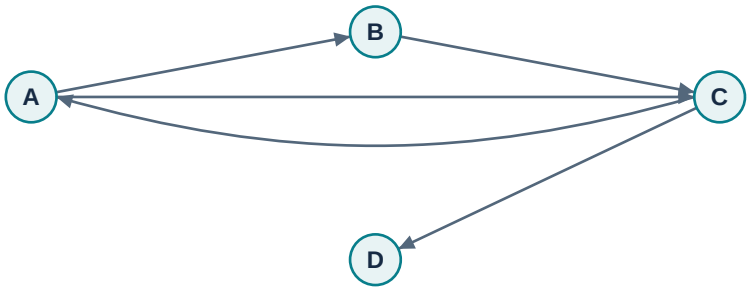
A. [7, 3, 2, 1 | 8, 9]
B. [7, 1, 2, 3 | 8, 9]
C. [3, 7, 2, 1 | 8, 9]
D. [8, 7, 2, 1 | 3, 9]

TRACE / WORKING

14 Graph representations and degrees

WEEK 4

Use this directed simple graph. M[u,v]=1 iff u->v exists; rows and columns are A,B,C,D. No edges beyond those shown.



Edges: A->B, A->C, B->C, C->A, C->D

Q40 / EASY

What are indegree(C) and outdegree(C)?

A. (2, 1)
B. (1, 3)
C. (3, 1)
D. (2, 2)

Q41 / MEDIUM

What is row C of the adjacency matrix, and how many adjacency-list entries are there in total?

A. [1,1,0,1]; 6 entries
B. [1,0,0,1]; 5 entries
C. [1,1,0,0]; 5 entries
D. [1,0,0,1]; 10 entries

Q42 / HARD

Now ignore directions and merge the two A/C arrows into one undirected edge. What are m, the degree sum, and the total adjacency-list entries of the resulting simple graph?

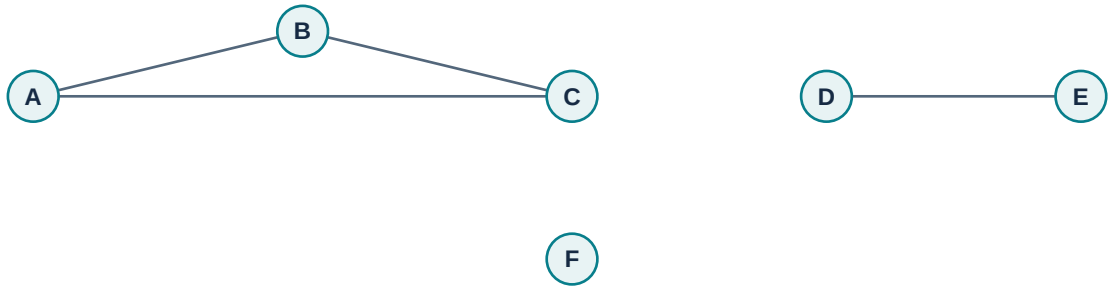
A. m=4; degree sum=8; 4 list entries
B. m=5; degree sum=10; 10 list entries
C. m=4; degree sum=8; 8 list entries
D. m=4; degree sum=4; 4 list entries

TRACE / WORKING

15 Paths, induced subgraphs and components

WEEK 4

Use the undirected graph shown. F is isolated. Path length counts edges; the induced graph G[U] retains every original edge with both endpoints in U.



Edges: A-B, B-C, A-C, D-E

Q43 / EASY

What is the connected-component partition?

A. {A,C}, {B,D,E}, {F}
B. {A,B}, {C}, {D,E}, {F}
C. {A,B,C}, {D,E}, {F}
D. {A,B,C,D,E}, {F}

Q44 / MEDIUM

What edges occur in G[{A,C,D,F}], and what is the original graph distance d(A,C)?

A. AB and BC; d(A,C)=1
B. Only AC; d(A,C)=1
C. No edges; d(A,C)=2
D. AC and CD; d(A,C)=2

Q45 / HARD

Delete edge AB, then add edge CD. Which describes the resulting graph?

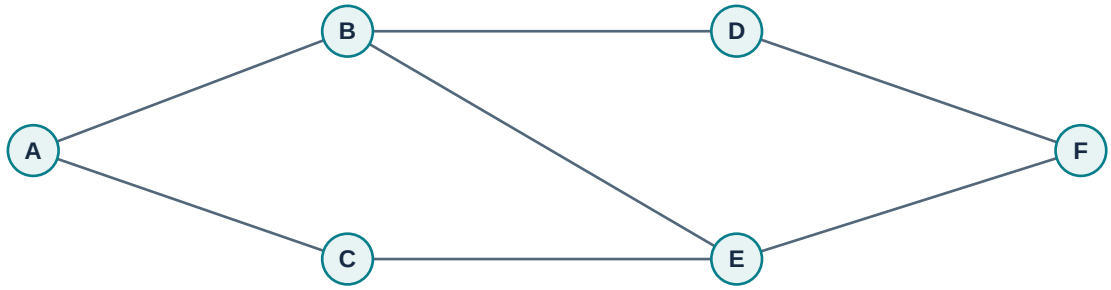
A. One component; the whole graph is a tree.
B. Two components; {A,B,C,D,E} still contains a cycle.
C. Two components; {A,B,C,D,E} is a tree and {F} is isolated.
D. Three components; deleting AB disconnected A from C.

TRACE / WORKING

16 Breadth-first search (BFS)

WEEK 4

Start BFS at A. Scan neighbours alphabetically; mark a vertex when enqueued and never enqueue it again. Queue is written front-to-back. Each question restarts BFS.



Edges: A-B, A-C, B-D, B-E, C-E, D-F, E-F

Q46 / EASY

Immediately after A is dequeued and all its neighbours are processed, what is the queue?

A. [B,C,D,E]
B. [B,C]
C. [C,B]
D. [A,B,C]

Q47 / MEDIUM

Immediately after B is dequeued and fully processed, what are the queue and parent(E)?

A. [C,D,E]; parent(E)=C
B. [C,E,D]; parent(E)=B
C. [C,D,E]; parent(E)=B
D. [D,E,C]; parent(E)=B

Q48 / HARD

After BFS finishes, what are parent(F), d(F), and the queue immediately after C was fully processed?

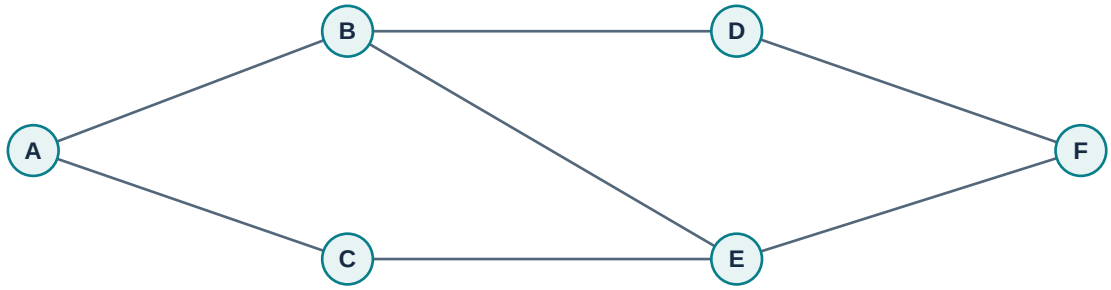
A. parent(F)=E; d(F)=3; queue=[D,E,E]
B. parent(F)=D; d(F)=2; queue=[D,E]
C. parent(F)=D; d(F)=3; queue=[D,E]
D. parent(F)=E; d(F)=3; queue=[E,D]

TRACE / WORKING

17 Recursive depth-first search (DFS)

WEEK 4

Use the same undirected graph drawn here. Start recursive DFS at A; scan neighbours alphabetically and mark on entry. Stack is active recursive calls, bottom-to-top.



Edges: A-B, A-C, B-D, B-E, C-E, D-F, E-F

Q49 / EASY

What are the first four discoveries?

- **A.** [A,B,C,D]
- **B.** [A,B,D,F]
- **C.** [A,C,E,F]
- **D.** [A,B,E,C]

Q50 / MEDIUM

What is the active call stack immediately after E is first discovered?

- **A.** [A,B,D,E,F]
- **B.** [A,C,E]
- **C.** [A,B,D,F,E]
- **D.** [A,B,E]

Q51 / HARD

Which gives the full finishing order and parent(C)?

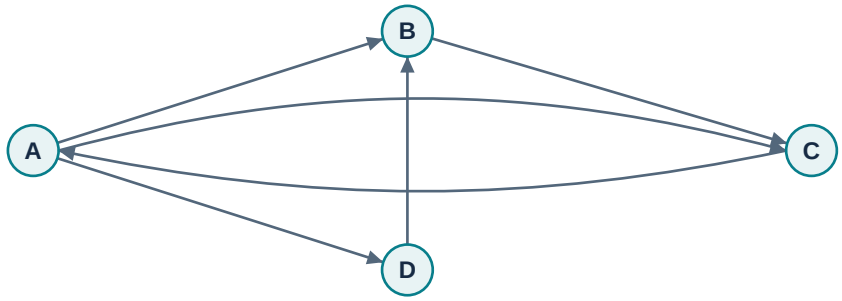
- **A.** [A,B,D,F,E,C]; parent(C)=A
- **B.** [C,E,F,D,B,A]; parent(C)=A
- **C.** [C,E,F,D,B,A]; parent(C)=E
- **D.** [C,E,D,F,B,A]; parent(C)=E

TRACE / WORKING

18 DFS tree, forward, back and cross edges

WEEK 4

Directed DFS: start A; scan outgoing neighbours alphabetically. Edges are A->B, A->C, A->D, B->C, C->A, D->B. Classify tree edges before applying non-tree rank tests.



Edges: A->B, B->C, C->A, A->D, A->C, D->B

Q52 / EASY

How is C->A classified?

- **A.** Tree edge
- **B.** Cross edge
- **C.** Back edge
- **D.** Forward edge

Q53 / MEDIUM

How are A->C and D->B classified, respectively?

- **A.** Cross; forward
- **B.** Forward; back
- **C.** Tree; back
- **D.** Forward; cross

Q54 / HARD

Use separate preorder/postorder ranks: A=(1,4), B=(2,2), C=(3,1), D=(4,3). In edge order AB, BC, CA, AD, AC, DB, which classification is correct?

- **A.** Tree, tree, back, tree, cross, forward
- **B.** Tree, tree, back, tree, forward, cross
- **C.** Tree, tree, cross, tree, forward, back
- **D.** Forward, forward, back, forward, forward, cross

TRACE / WORKING

19 DFS cycle detection and full forests

WEEK 4

Use the directed graph shown for Easy and Medium. Full DFS starts new roots alphabetically whenever an unvisited vertex remains; outgoing neighbours are alphabetical.



Edges: A->B, C->D, D->E, E->C

Q55 / EASY

Run DFS only from A. What does this search establish about cycles?

- **A.** The entire graph is acyclic because no back edge was found.
- **B.** The entire graph has a cycle because AB is a tree edge.
- **C.** It visits C,D,E before returning from A.
- **D.** The reachable subgraph {A,B} is acyclic; it does not decide acyclicity of the whole graph.

Q56 / MEDIUM

Continue to a full DFS forest. Which edge first certifies a directed cycle, and what is the witness cycle?

- **A.** E->C; C->D->E->C
- **B.** A->B; A->B->A
- **C.** D->E; D->E->D
- **D.** C->D; C->D->E

Q57 / HARD

In a different SIMPLE UNDIRECTED graph with edges AB, BC, CA, DFS follows tree edges AB then BC. Which scan is a valid cycle certificate?

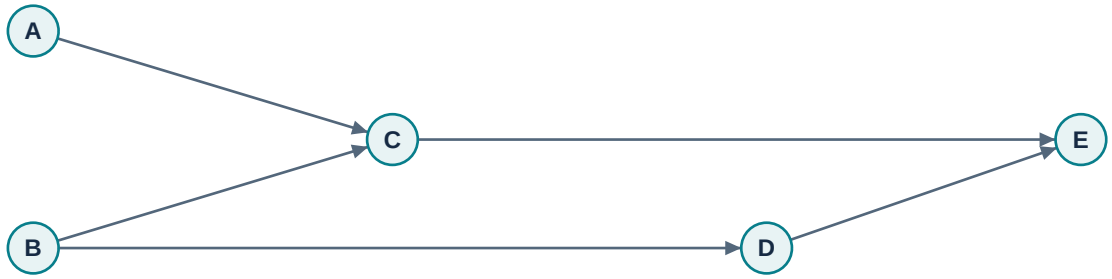
- **A.** At C, seeing parent B; only reverse parent scans certify cycles.
- **B.** At C, seeing visited non-parent A; the reverse scan B->A alone is not a certificate.
- **C.** No cycle can exist because DFS tree edges form a tree.
- **D.** At B, seeing parent A; every edge to a visited vertex certifies a cycle.

TRACE / WORKING

20 DAGs and source-removal topological sort

WEEK 5

Use this DAG. An arrow u->v means u must precede v. Source removal always chooses the alphabetically smallest CURRENT zero-indegree vertex, updating the available set after each removal.



Edges: A->C, B->C, B->D, C->E, D->E

Q58 / EASY

Which vertices are initial sources, and which are initial sinks?

- **A.** Sources {E}; sinks {A,B}
- **B.** Sources {A}; sinks {D,E}
- **C.** Sources {A,B}; sinks {E}
- **D.** Sources {A,B}; sinks {C,D}

Q59 / MEDIUM

After removing A only, what are indegree(C) and the current available zero-indegree set?

- **A.** indegree(C)=0; available={B,C}
- **B.** indegree(C)=2; available={B}
- **C.** indegree(C)=1; available={B,D}
- **D.** indegree(C)=1; available={B}

Q60 / HARD

Which is the algorithm output, and is this graph's topological order unique?

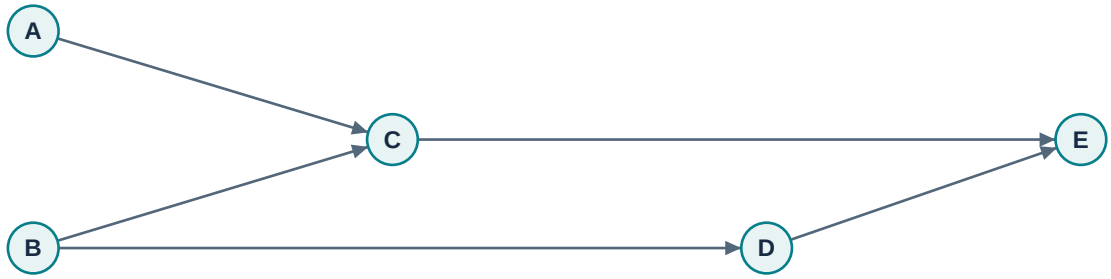
- **A.** [A,B,C,D,E]; unique
- **B.** [B,D,E,A,C]; unique
- **C.** [A,B,C,D,E]; not unique
- **D.** [A,C,B,D,E]; not unique

TRACE / WORKING

21 DFS topological sort and partial orders

WEEK 5

Use the same DAG shown here. For full DFS, consider roots alphabetically, scan outgoing neighbours alphabetically, and output reverse finishing order. Reachability is taken reflexively for the partial order.



Edges: A->C, B->C, B->D, C->E, D->E

Q61 / EASY

DFS finishes vertices in order [E,C,A,D,B]. Which topological order does it output?

A. [B,D,A,C,E]
B. [E,C,A,D,B]
C. [B,D,E,A,C]
D. [A,C,E,B,D]

Q62 / MEDIUM

With the stated DFS rules, what are the first three FINISHED vertices?

A. [E,C,A]
B. [C,E,A]
C. [E,D,C]
D. [A,C,E]

Q63 / HARD

Under reflexive reachability, which pair is incomparable, and which added arrow would destroy acyclicity?

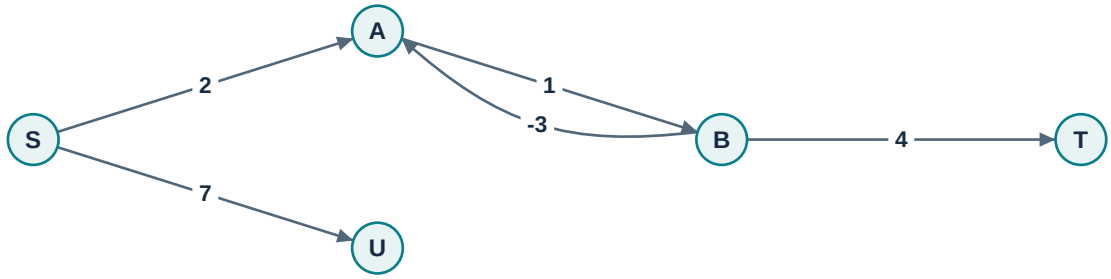
A. A and E are incomparable; adding E->B preserves acyclicity.
B. C and D are incomparable; adding A->E creates a cycle.
C. C and D are incomparable; adding E->B creates a cycle.
D. B and E are incomparable; adding D->C creates a cycle.

TRACE / WORKING

22 Shortest paths, relaxation and negative cycles

WEEK 6

Use the directed weighted graph shown where specified. INF denotes unreachable; -INF denotes distances unbounded below. Relaxation changes a label and parent only on a strict improvement.



Edges: S->A(2), A->B(1), B->A(-3), B->T(4), S->U(7)

Q64 / EASY

Independently of the diagram, d[u]=3, d[v]=9, parent[v]=x, and w(u,v)=2. What does Relax(u,v) do?

A. Set d[v]=5 and parent[v]=u.
B. Set d[v]=2 and parent[v]=u.
C. Keep d[v]=9 and parent[v]=x.
D. Set d[v]=5 and parent[v]=x.

Q65 / MEDIUM

In the diagram, what are the shortest-distance statuses of T and U from S?

A. Both T and U are -INF.
B. T is -INF; U has finite distance 7.
C. T has finite distance 7; U has finite distance 7.
D. T is INF; U has finite distance 7.

Q66 / HARD

In the diagram, remove edge B->A. Which is a shortest S-to-T route, and what distance label must hold for B on that shortest route?

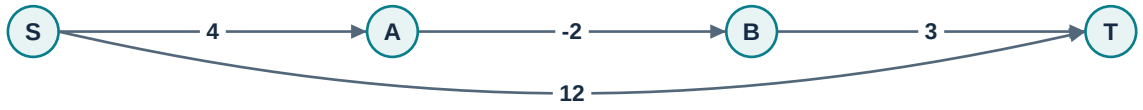
A. S->A->B->T, weight 7; d[B]=3.
B. S->U->T, weight 11; d[B]=3.
C. No shortest route exists because the original graph had a negative cycle.
D. S->A->B->T, weight 7; d[B]=1.

TRACE / WORKING

23 Bellman-Ford passes and detection

WEEK 6

Start at S; distances ordered (S,A,B,T). Each complete pass scans edges in this exact order: B->T(3), A->B(-2), S->A(4), S->T(12). Update distances IN PLACE; initially (0,INF,INF,INF).



Edges: S->A(4), A->B(-2), B->T(3), S->T(12)

Q67 / EASY

What is the distance vector after pass 1?

- **A.** (0,4,2,12)
- **B.** (0,4,2,5)
- **C.** (0,4,INF,12)
- **D.** (0,INF,INF,12)

Q68 / MEDIUM

What is the distance vector after pass 2?

- **A.** (0,4,2,5)
- **B.** (0,4,-2,1)
- **C.** (0,4,INF,12)
- **D.** (0,4,2,12)

Q69 / HARD

After the required |V|-1=3 passes, which statement about this graph and the additional detection scan is correct?

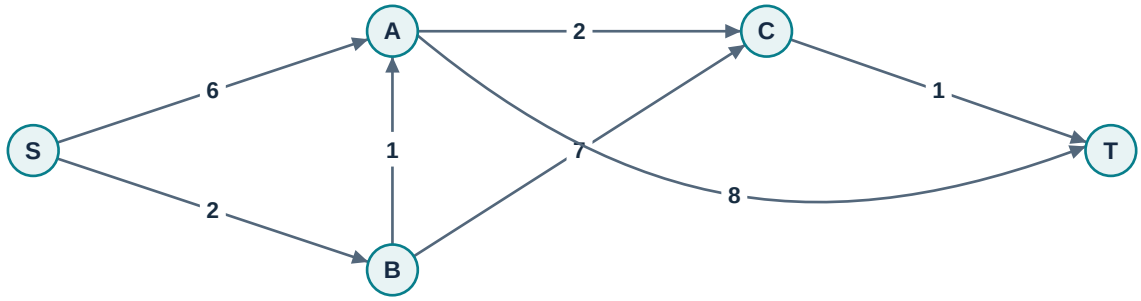
- **A.** Distances are (0,4,2,5); an extra-scan improvement would prove that improving edge itself lies on a negative cycle.
- **B.** Distances are (0,4,2,5); no edge improves, so no reachable negative cycle is detected.
- **C.** Distances are (0,4,2,12); the negative edge proves a negative cycle.
- **D.** Distances are (0,4,2,5); every negative edge must improve forever.

TRACE / WORKING

24 Dijkstra: labels, parents and settling

WEEK 6

Start at S. Settle a minimum tentative-distance unsettled vertex, then relax all outgoing edges. Break extraction ties alphabetically; use strict improvements. Each question starts a fresh run.



Edges: S->A(6), S->B(2), B->A(1), B->C(7), A->C(2), A->T(8), C->T(1)

Q70 / EASY

After settling S and relaxing its outgoing edges, which vertex is settled next, and with what distance?

- **A.** C with distance 9
- **B.** B with distance 2
- **C.** T with distance INF
- **D.** A with distance 6

Q71 / MEDIUM

After settling S, B, A and relaxing A's outgoing edges, which are d[C], d[T], and parent(C)?

- **A.** d[C]=5; d[T]=6; parent(C)=A
- **B.** d[C]=5; d[T]=11; parent(C)=A
- **C.** d[C]=9; d[T]=11; parent(C)=B
- **D.** d[C]=2; d[T]=8; parent(C)=A

Q72 / HARD

Which full settling order and final shortest route to T are correct?

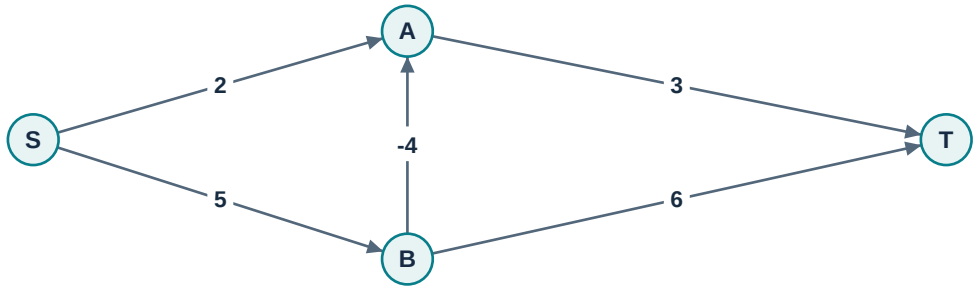
- **A.** [S,B,A,T,C]; S->B->A->T, weight 11
- **B.** [S,B,A,C,T]; S->B->A->C->T, weight 6
- **C.** [S,B,A,C,T]; S->B->C->T, weight 10
- **D.** [S,A,B,C,T]; S->A->T, weight 14

TRACE / WORKING

25 DAG shortest paths and algorithm selection

WEEK 6

Recording-mentioned tutorial extension: topologically order a DAG, initialise source labels, then relax outgoing edges once in that order. Use order [S,B,A,T] on the diagram.



Edges: S->A(2), S->B(5), B->A(-4), A->T(3), B->T(6)

Q73 / EASY

Immediately after processing S in the DAG algorithm, what are (d[A],d[B],d[T])?

A. (1,5,4)
B. (2,5,5)
C. (2,5,INF)
D. (2,INF,INF)

Q74 / MEDIUM

What are the final shortest distance to T and its route?

A. 5 via S->A->T
B. 11 via S->B->T
C. No finite distance exists because BA is negative.
D. 4 via S->B->A->T

Q75 / HARD

On this graph, standard Dijkstra never reopens settled vertices. Which comparison is correct?

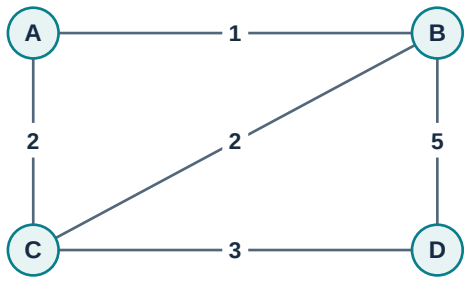
A. Dijkstra can settle A too early at 2; topological relaxation returns 4 for T in O(V+E).
B. The DAG algorithm must use Bellman-Ford's V-1 complete passes.
C. A negative edge prevents every shortest-path algorithm from working.
D. Dijkstra is guaranteed correct because there is no negative cycle.

TRACE / WORKING

26 MST objective, cuts and cycle property

WEEK 7

Use the connected undirected weighted graph shown. A cut consists of every edge crossing a partition; an MST minimises total chosen edge weight.



Edges: A-B(1), A-C(2), B-C(2), B-D(5), C-D(3)

Q76 / EASY

For cut {A,B} | {C,D}, which are exactly the crossing edges?

A. {AB,CD}
B. {AB,AC,BC,BD,CD}
C. {AC,BC,CD}
D. {AC,BC,BD}

Q77 / MEDIUM

In cycle B-C-D-B, which edge is excluded from EVERY MST by the strict cycle property?

A. CD, weight 3
B. No edge, because cycle edges can always appear in every MST.
C. BC, weight 2
D. BD, weight 5

Q78 / HARD

Which describes all MST possibilities and their total weight?

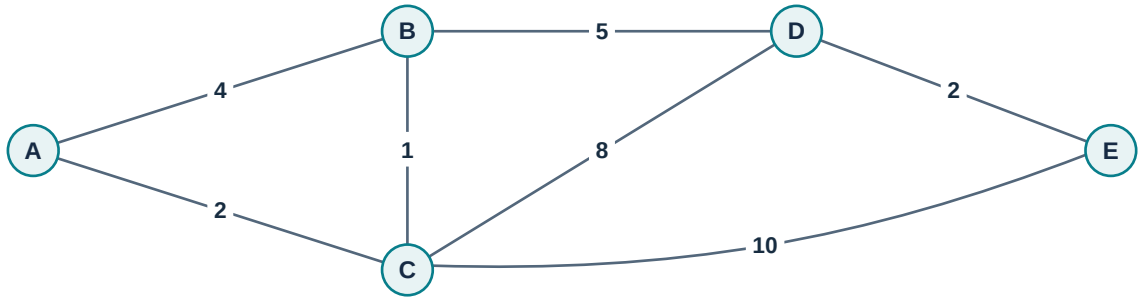
A. Exactly two MSTs, each weight 6: {AB,AC,CD} or {AB,BC,CD}.
B. Three MSTs, including {AC,BC,CD}, all weight 6.
C. One MST, weight 6: {AB,AC,CD} only.
D. Exactly two MSTs, each weight 8: {AB,AC,BD} or {AB,BC,BD}.

TRACE / WORKING

27 Prim: cheapest attachment edges

WEEK 7

Start Prim at A. key[v] is the cheapest SINGLE edge from the current tree to outside vertex v. Break equal keys alphabetically; change parent only on strict improvement.



Edges: A-B(4), A-C(2), B-C(1), B-D(5), C-D(8), C-E(10), D-E(2)

Q79 / EASY

Which edge is added first, and what key does it use?

A. DE, key 2
B. BC, key 1
C. AB, key 4
D. AC, key 2

Q80 / MEDIUM

After A,C,B have been added and their edges processed, what are the keys of D and E, and which edge is added next?

A. key[D]=5; key[E]=2; next DE
B. key[D]=8; key[E]=10; next CD
C. key[D]=8; key[E]=12; next CD
D. key[D]=5; key[E]=10; next BD

Q81 / HARD

Which gives the final MST edges, total weight, and final parent(E)?

A. {AC,CB,CD,DE}; weight 13; parent(E)=D
B. {AC,CB,BD,DE}; weight 10; parent(E)=D
C. {AC,AB,BD,DE}; weight 13; parent(E)=D
D. {AC,CB,BD,CE}; weight 18; parent(E)=C

TRACE / WORKING

28 Union-find: naive linking and compression

WEEK 7

Parents are written p[x]; a root points to itself. In the naive Union(x,y), attach root(x) BELOW root(y). Compression, where specified, makes each vertex on the find path point directly to its root.

Q82 / EASY

With p[A]=B, p[B]=C, p[C]=C, what does Find(A) return without compression?

A. C
B. B
C. The smallest key A
D. A

Q83 / MEDIUM

Start singleton sets {A},{B},{C},{D}. Without compression, perform Union(A,B), Union(A,C), Union(A,D). What are parents in order (A,B,C,D)?

A. (B,C,D,D)
B. (D,D,D,D)
C. (B,B,D,D)
D. (A,A,A,A)

Q84 / HARD

Start p=(B,C,D,D,D) for vertices (A,B,C,D,E). Call Find(B) with full path compression. What are the resulting parents, and does the set partition change?

A. (B,D,D,D,E); E becomes a singleton
B. (B,D,D,D,D); partition unchanged
C. (B,C,D,D,D); compression cannot change parents
D. (D,D,D,D,D); partition unchanged

TRACE / WORKING

29 Union by rank with path compression

WEEK 7

Initial singleton ranks are 0. Union finds roots, links lower rank below higher rank, and on EQUAL rank links the FIRST root below the SECOND and increments the second root's rank. Find fully compresses its own path.

Q85 / EASY

From singleton A and B, perform Union(A,B). Which is correct?

A. p[B]=A; rank[A]=1

B. p[A]=B; rank[B]=0

C. p[A]=A; p[B]=B; both ranks become 1

D. p[A]=B; rank[B]=1

Q86 / MEDIUM

From singleton A,B,C,D, perform Union(A,B), Union(C,D), Union(B,D). Which parents and root rank result?

A. p=(B,D,D,D); rank[D]=2

B. p=(B,B,D,B); rank[B]=2

C. p=(B,D,D,D); rank[D]=1

D. p=(D,D,D,D); rank[D]=2

Q87 / HARD

Continue the Medium state: Find(A), then Union(A,C). Which result is correct?

A. p=(D,D,D,D); rank[D] falls to 1.

B. p=(D,D,D,D); rank[D] rises to 3.

C. p=(B,D,D,D); rank[D] remains 2.

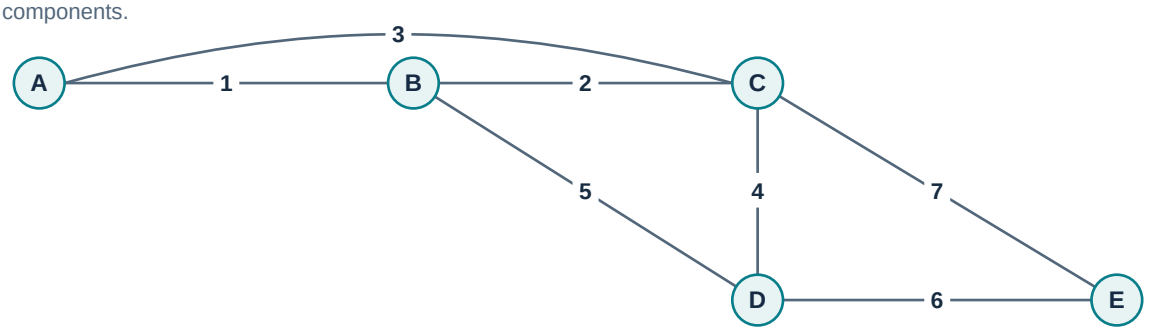
D. p=(D,D,D,D); rank[D] remains 2 even though the actual tree height is 1.

TRACE / WORKING

30 Kruskal: accepted edges and forest state

WEEK 7

Sort undirected edges by increasing weight. Accept an edge iff its endpoints are in distinct union-find components. Each question starts with all singleton components.



Edges: A-B(1), B-C(2), A-C(3), C-D(4), B-D(5), D-E(6), C-E(7)

Q88 / EASY

What are the accepted edges after examining the first two sorted edges?

- **A.** {AB} only
- **B.** {AB,AC}
- **C.** {BC,CD}
- **D.** {AB,BC}

Q89 / MEDIUM

After examining the first FOUR sorted edges, which edges were accepted, which rejected, and what components remain?

- **A.** Accept AB,AC,CD; reject BC; components {A,B,C,D}, {E}.
- **B.** Accept AB,BC; reject AC,CD; components {A,B,C}, {D}, {E}.
- **C.** Accept AB,BC,CD; reject AC; components {A,B,C,D}, {E}.
- **D.** Accept AB,BC,AC,CD; reject none; components {A,B,C,D}, {E}.

Q90 / HARD

Stop at |V|-1 accepted edges. What is the complete accepted set, its total weight, and how many sorted edges were examined?

- **A.** {AB,BC,CD,DE}; weight 13; 4 edges examined
- **B.** {AB,BC,BD,DE}; weight 14; 5 edges examined
- **C.** {AB,BC,CD,DE}; weight 13; 6 edges examined
- **D.** {AB,AC,CD,DE}; weight 14; 6 edges examined

TRACE / WORKING

Answers and explanations

01 Sorting records and stability

Q01 - D (Easy)
[1x, 2y, 3a, 3b]
Keys become 1,2,3,3; equal-key records 3a and 3b retain their input order. Record identities cannot be detached from keys.

Q02 - C (Medium)
Number first, then letter; [(A,1), (A,3), (B,1), (B,2)]
Sort the secondary key first. The subsequent stable primary-key sort preserves ascending numbers within each letter group.

Q03 - A (Hard)
It produced a sorted permutation, but the sorter is not stable.
The output contains exactly the input records and nondecreasing keys. Reversing 2a and 2b disproves stability. An output trace alone establishes no space bound.

Source: 01 Sorting, pp. 3-6; recording S1, 08:02-14:55.

02 Insertion sort

Q04 - B (Easy)
[3, 7, 5, 2]
Insert 3 into the sorted prefix [7] by one adjacent swap; the unprocessed suffix [5,2] stays in place.

Q05 - C (Medium)
[3, 5, 7, 2]
After i=1 the prefix is [3,7]. At i=2, swap 5 with 7 and stop at 3; only the first three entries are sorted.

Q06 - B (Hard)
5 swaps; [1, 2a, 2b, 4]
The iterations make 1,1,3 swaps. Equal keys never swap under strict >, so 2a stays before 2b. The five strict inversions each require one swap.

Source: 01 Sorting, pp. 8-9; recording S2, 12:58-19:32.

Answers and explanations

03 Merge sort and stable merging

Q07 - A (Easy)
[3, 8, 6, 1]
The left recursive call sorts indices [0,2). The right half [6,1] has not yet been processed.

Q08 - A (Medium)
[1a, 1b, 3, 4a]
Choose left 1a on the first tie, then right 1b and 3. The next 4 tie selects left 4a.

Q09 - C (Hard)
[1,5] and [2,3,4]; 4 comparisons
mid=2, so the halves have sizes 2 and 3. Compare 1/2, 5/2, 5/3, 5/4. Once the right half is exhausted, copy 5 without a front-key comparison.

Source: 01 Sorting, pp. 11-15; recording S3, 08:46-11:29.

04 Quicksort: one-scan partition

Q10 - A (Easy)
[2, 1, 4, 6, 5]; j=2
Only 2 and 1 move into the smaller region. The final pivot swap exchanges 4 with 5 at index 2; partition does not sort each side.

Q11 - A (Medium)
[2, 6, 5, 1, 4]; j=1
i=0 sees 6 and does nothing; i=1 swaps 2 with 6 and increments j to 1; i=2 sees 5 and does nothing.

Q12 - A (Hard)
[2, 1, 4c, 4a, 4b]; [0,1] and [3,4]
Strict < moves only 2 and 1 before the pivot. Before the final swap A=[2,1,4b,4a,4c]; swapping indices 2 and 4 settles 4c at 2. Exclude the settled pivot from both recursions.

Source: 01 Sorting, pp. 17-20; recording S4, 09:18-12:08.

Answers and explanations

05 Partition alternatives and pivot choice

Q13 - A (Easy)
[1, 2, 3, 6, 7]
The left scan stops at 7 and the right scan at 2. Exchange those two misplaced values; no final pivot-position swap is specified here.

Q14 - B (Medium)
< block contains {1,2}; = block {4a,4b,4c}; > block {6}; recurse only on < and >.
All keys equal to the pivot are already in their final key block. Only the strictly smaller/larger blocks require recursion; no within-block record order is promised.

Q15 - A (Hard)
(n-1,0), then (n-2,0), ...; n(n-1)/2 comparisons
The pivot is always the largest remaining key. Each partition scans all other keys, giving (n-1)+(n-2)+...+1 comparisons and quadratic time.

Source: 01 Sorting, pp. 21-24; recording S4, 12:12-15:05, 21:19-23:24.

06 Counting sort

Q16 - A (Easy)
[1, 2, 2, 1]
There is one zero, two ones, two twos, and one three. C indexes keys, rather than input positions.

Q17 - D (Medium)
C=[1,3,5,6]; first put 1b at B[2]
C[1]=3 initially. The last input record is 1b; decrement C[1] to 2, then write B[2]. Counts and last-index conventions must not be mixed.

Q18 - D (Hard)
B=[_,1,1b,_,_,3]; C=[1,1,5,5]
Place 1b at 2, then 3 at 5, then the earlier record 1 at 1. The updated counts are [1,1,5,5]; this backward placement preserves equal-key order.

Source: 02 Linear Time Sorting, pp. 5-20; recording S5, 23:53-24:05.

Answers and explanations

07 LSD radix sort

Q19 - B (Easy)
[720, 355, 436, 457, 657, 329, 839]
Units are 0,5,6,7,7,9,9 in output order. Stability keeps 457 before 657 and 329 before 839.

Q20 - C (Medium)
[720, 329, 436, 839, 355, 457, 657]
Sorting the units-pass output by tens gives digit groups 2:{720,329}, 3:{436,839}, 5:{355,457,657}. The last two digits are now ordered.

Q21 - D (Hard)
It produces [12,11,22,21], so the two-pass result is not numerically sorted.
The tens groups before reversal are [11,12] and [21,22]. Reversing them destroys units order inside each group, invalidating the LSD invariant.

Source: 02 Linear Time Sorting, pp. 22-23; recording S6, 02:30-08:22.

08 Sorting cost, space and lower bounds

Q22 - C (Easy)
Heapsort using iterative heap repair
Iterative max-heap repair allows heapsort to work within the input array. The other specified implementations allocate nonconstant extra storage.

Q23 - B (Medium)
Both are Theta(n^2).
The key-range size k is n^2. Initialising/scanning that count array gives Theta(n+k) time and the output-plus-count arrays give Theta(n+k) space.

Q24 - A (Hard)
At most 3 passes; O(n) time. This does not contradict the comparison-sort lower bound.
The range fits three base-n digits. Each pass costs O(n+n), so three passes cost O(n). Digit extraction and count indexing use the integer model, outside the comparison-only lower bound.

Source: 01 Sorting, pp. 6,15,23-24; 02 Linear Time Sorting, pp. 2-3,19,23; 03 Heaps, pp. 42-48.

Answers and explanations

09 Heap shape, order and indexing

Q25 - A (Easy)
Parent 1; children 7 and 8
For i=3, parent=floor((i-1)/2)=1, left=2i+1=7, right=2i+2=8. Both children are within 0..9.

Q26 - D (Medium)
[1, 4, 2, 8, 5, 3]
The valid array satisfies 1<=4,2; 4<=8,5; 2<=3. Siblings need not be ordered. Each other array has a child smaller than its parent.

Q27 - B (Hard)
Leaves are 5..10; at least one leaf contains a largest key, but its exact index is not fixed.
Internal indices are 0..floor(11/2)-1=4. A maximum internal value can be followed down through children of no smaller value to a leaf. Heap order does not order unrelated branches.

Source: 03 Heaps, pp. 3-6; recording H1, 06:38-12:03.

10 Min-Heapify: repair downward

Q28 - B (Easy)
[3, 9, 5, 7, 8, 6]
At root 9, the smaller child is 3 at index 1. Swap only indices 0 and 1 for this first repair step.

Q29 - B (Medium)
[3, 7, 5, 9, 8, 6]
After the first swap, key 9 at index 1 has children 7 and 8. Swap with 7 and stop at leaf index 3.

Q30 - D (Hard)
The array stays unchanged; the whole array is still not a min-heap.
Root 1 is already <=9 and 2, so the call stops. The invalid child subtree rooted at 9 is never repaired; the required child-subtree precondition was false.

Source: 03 Heaps, pp. 9-16; recording H2, 11:51-14:05.

Answers and explanations

11 Bottom-up Build-Min-Heap

Q31 - D (Easy)
[9, 4, 2, 1, 3, 6, 7]; first index 2
The last internal node is floor(7/2)-1=2. Its children are 6 and 2, so swap 7 with 2 at index 6.

Q32 - D (Medium)
[9, 1, 2, 4, 3, 6, 7]
At index 1 the key is 4 and the children are 1 and 3. Swap 4 with 1; the right subtree from the previous call remains [2,6,7].

Q33 - A (Hard)
[1, 3, 2, 4, 9, 6, 7]; Theta(n) construction time
At root, move 9 below 1 and then below 3. Bottom-up construction sums work over subtree heights: many nodes are leaves or near leaves, giving Theta(n), rather than n full-height repairs.

Source: 03 Heaps, pp. 17-28; recording H2, 15:18-18:36, 25:32-26:45.

12 Heap-based priority queue

Q34 - C (Easy)
[1, 5, 2, 9, 7, 8, 3]
Append 1 at index 6; swap with parent 3 at index 2, then with root 2. Other positions do not move.

Q35 - D (Medium)
Return 2; [3, 5, 8, 9, 7]
Save 2. Move last key 8 to root and shrink to five entries; swap it with smaller child 3. Index 2 is now a leaf.

Q36 - C (Hard)
Returns 2, then 3; [4, 5, 8, 9, 7]
Insertion yields [2,5,3,9,7,8,4]. First extraction yields [3,5,4,9,7,8]. Replacing 3 by 8 and repairing yields [4,5,8,9,7]. Priority order differs from arrival order.

Source: 03 Heaps, pp. 30-40; recording H3, 02:32-09:36.

Answers and explanations

13 Heapsort and the active boundary

Q37 - B (Easy)
[2, 7, 8, 1, 3, 9]
Swap A[0]=9 with A[5]=2. This question stops before the repair that would move 8 to the root.

Q38 - B (Medium)
[8, 7, 2, 1, 3 | 9]
The active heap has five entries. Swap root 2 with its larger child 8 at index 2. The fixed 9 is excluded from repair.

Q39 - A (Hard)
[7, 3, 2, 1 | 8, 9]
From [8,7,2,1,3|9], swap 8 with 3, shrink to four, then swap root 3 with child 7. Active [7,3,2,1] remains a max-heap; suffix [8,9] is sorted.

Source: 03 Heaps, pp. 42-48; recording H4, 01:15-04:19.

14 Graph representations and degrees

Q40 - D (Easy)
(2, 2)
A->C and B->C enter C. C->A and C->D leave C. The reciprocal A/C arrows are distinct directed edges.

Q41 - B (Medium)
[1,0,0,1]; 5 entries
Row C records outgoing neighbours A and D. A directed graph stores one outgoing-list entry per directed edge, so all lists together contain 5 entries.

Q42 - C (Hard)
m=4; degree sum=8; 8 list entries
The remaining undirected edges are AB,AC,BC,CD. Each contributes two to the degree sum and two stored neighbour entries. Reciprocal arrows collapse to one edge.

Source: 04 Intro to Graphs, pp. 11-17; recording G1, 27:23-31:21.

Answers and explanations

15 Paths, induced subgraphs and components

Q43 - C (Easy)
{A,B,C}, {D,E}, {F}
A,B,C form one connected triangle, D/E form one connected edge, and isolated F forms its own maximal connected component.

Q44 - B (Medium)
Only AC; d(A,C)=1
Both endpoints of AC are retained. B and E are deleted, so their incident edges vanish. Original A/C have a direct edge, hence distance 1.

Q45 - C (Hard)
Two components; {A,B,C,D,E} is a tree and {F} is isolated.
Deleting AB breaks the triangle but leaves A-C-B connected. CD joins it to D-E, making five vertices with four connected, acyclic edges. F remains separate.

Source: 04 Intro to Graphs, pp. 10,18-23; recording G1, 33:17-41:40.

16 Breadth-first search (BFS)

Q46 - B (Easy)
[B,C]
A discovers B then C. A has already been removed; D/E are not discovered until later expansions.

Q47 - C (Medium)
[C,D,E]; parent(E)=B
Before processing B, Q=[B,C]. B discovers D then E, appending them after C. E is marked immediately, with B as parent.

Q48 - C (Hard)
parent(F)=D; d(F)=3; queue=[D,E]
C sees already-discovered E and adds nothing. D is expanded before E and first discovers F. The tree route is A-B-D-F, with three edges.

Source: 05 Graph Search, pp. 4-16; recording G2, 03:09-03:46, 18:30-18:46.

Answers and explanations

17 Recursive depth-first search (DFS)

Q49 - B (Easy)
[A,B,D,F]
From A choose B; from B choose D after skipping visited A; from D go to F. DFS follows a branch before returning.

Q50 - C (Medium)
[A,B,D,F,E]
F is reached through D and next recursively enters E. All ancestors remain active until their calls return; this differs from discovery order alone.

Q51 - C (Hard)
[C,E,F,D,B,A]; parent(C)=E
E reaches C, which has no unvisited neighbour and finishes first. Then finish E,F,D,B,A as recursion unwinds. Although AC exists, C was discovered earlier through E.

Source: 05 Graph Search, pp. 18-29; recording G3, 12:04-15:28.

18 DFS tree, forward, back and cross edges

Q52 - C (Easy)
Back edge
DFS enters A,B,C. When C scans A, A is an active ancestor, so C->A is a back edge.

Q53 - D (Medium)
Forward; cross
B first discovers C, so AC is a non-tree ancestor-to-descendant edge. By the time D is entered, B is finished in a different branch; DB is cross.

Q54 - B (Hard)
Tree, tree, back, tree, forward, cross
The parent edges are AB,BC,AD. For non-tree CA, preorder decreases and postorder increases (back); AC goes to a descendant (forward); DB decreases both ranks (cross). Ranks are separate 1..4 counters.

Source: 05 Graph Search, pp. 30-34; recording G3, 16:01-21:21.

Answers and explanations

19 DFS cycle detection and full forests

Q55 - D (Easy)
The reachable subgraph {A,B} is acyclic; it does not decide acyclicity of the whole graph.
A reaches only B. The separate component containing C,D,E is unvisited, so a single-source search cannot rule out a cycle there.

Q56 - A (Medium)
E->C; C->D->E->C
After finishing A and B, start C, then enter D,E. E->C reaches an active ancestor; the tree path C-D-E plus EC closes the cycle.

Q57 - B (Hard)
At C, seeing visited non-parent A; the reverse scan B->A alone is not a certificate.
Each undirected tree edge is scanned from both ends. Ignore the immediate parent edge. The non-parent CA closes triangle A-B-C-A.

Source: 05 Graph Search, pp. 35-40; recording G3, 21:21-25:15.

20 DAGs and source-removal topological sort

Q58 - C (Easy)
Sources {A,B}; sinks {E}
A/B have no incoming arrows; E has no outgoing arrows. C and D each have both incoming and outgoing arrows.

Q59 - D (Medium)
indegree(C)=1; available={B}
C loses incoming AC but still has BC. Only B is available; D still depends on B.

Q60 - C (Hard)
[A,B,C,D,E]; not unique
After A then B, both C and D are ready; choose C alphabetically, then D,E. Multiple initial sources already allow distinct valid orders, such as B,A,C,D,E.

Source: 05 Graph Search, pp. 39-44; recording G4, 21:47-25:41.

Answers and explanations

21 DFS topological sort and partial orders

Q61 - A (Easy)
[B,D,A,C,E]
Reverse the complete finishing order. Direct finishing order puts E before its prerequisites and is invalid under prerequisite-forward arrows.

Q62 - A (Medium)
[E,C,A]
A recursively enters C then E. E finishes first, then C, then A. The B-rooted part is considered later; root coverage matters.

Q63 - C (Hard)
C and D are incomparable; adding E->B creates a cycle.
Neither C reaches D nor D reaches C. Existing B-C-E (or B-D-E) plus E-B makes a cycle. Adding A-E is consistent with reachability; adding D-C remains acyclic.

Source: 05 Graph Search, pp. 40,43-44; recording G4, 03:03-08:18, 26:26-28:58.

22 Shortest paths, relaxation and negative cycles

Q64 - A (Easy)
Set d[v]=5 and parent[v]=u.
The candidate route has weight d[u]+w=3+2=5, smaller than 9. Both the distance and the parent change.

Q65 - B (Medium)
T is -INF; U has finite distance 7.
Reachable cycle A-B-A has total weight -2 and can reach T, so routes to T are arbitrarily cheap. No route goes from that cycle to U; SU is the only route to U.

Q66 - A (Hard)
S->A->B->T, weight 7; d[B]=3.
Removing BA destroys the negative cycle. The remaining route S-A-B-T costs 2+1+4=7; its prefix to B costs 3 and is shortest, illustrating optimal substructure. UT is not an edge.

Source: 06 Shortest Path, pp. 3-10; recording P1, 14:09-21:26.

Answers and explanations

23 Bellman-Ford passes and detection

Q67 - C (Easy)
(0,4,INF,12)
BT and AB have unreachable tails at their scan times. SA sets A=4, and ST sets T=12. Earlier edges are not revisited within this pass.

Q68 - D (Medium)
(0,4,2,12)
BT is scanned before B becomes finite. Then AB uses A=4 and sets B=2. T cannot benefit from B until the following pass.

Q69 - B (Hard)
Distances are (0,4,2,5); no edge improves, so no reachable negative cycle is detected.
Pass 3 propagates B=2 across BT, giving 5. All labels then satisfy relaxation. A negative edge alone is harmless; in general an extra-scan improvement proves a reachable negative cycle, but the improving edge may be downstream of it.

Source: 06 Shortest Path, pp. 13-14; recording P1, 38:42-44:09.

24 Dijkstra: labels, parents and settling

Q70 - B (Easy)
B with distance 2
S offers A=6 and B=2. B is the smaller unsettled label and is extracted next.

Q71 - B (Medium)
d[C]=5; d[T]=11; parent(C)=A
B improves A to 3 and offers C=9. A then improves C to 3+2=5 and offers T=3+8=11. C has not yet offered the eventual T=6.

Q72 - B (Hard)
[S,B,A,C,T]; S->B->A->C->T, weight 6
Settled distances are 0,2,3,5,6. C improves T from 11 to 6. Reconstructing parents gives T<-C<-A<-B<-S. The first route offered to a vertex need not be optimal.

Source: 06 Shortest Path, pp. 15-29; recording P2, 10:40-15:31.

Answers and explanations

25 DAG shortest paths and algorithm selection

Q73 - C (Easy)
(2,5,INF)
S directly offers A=2 and B=5. Neither A nor B has been processed, so no route to T has yet been propagated.

Q74 - D (Medium)
4 via S->B->A->T
Process B before A: B improves A from 2 to 1 using 5-4. Processing A then improves T to 1+3=4. Acyclicity allows negative edges without negative cycles.

Q75 - A (Hard)
Dijkstra can settle A too early at 2; topological relaxation returns 4 for T in O(V+E).
Dijkstra extracts A before B and commits the incorrect A=2. It can settle T at 5 before processing B. Topological order respects BA and propagates the correct value without repeated passes.

Source: 05 Graph Search, pp. 39-44; 06 Shortest Path, pp. 9-10; recording P1, 34:17-34:36 (tutorial extension).

26 MST objective, cuts and cycle property

Q76 - D (Easy)
{AC,BC,BD}
AC and BC join the two sides, as does BD. AB and CD stay entirely within their respective sides.

Q77 - D (Medium)
BD, weight 5
BD=5 is strictly heavier than BC=2 and CD=3 on that cycle; any spanning tree using it can be improved by exchanging across the resulting cut.

Q78 - A (Hard)
Exactly two MSTs, each weight 6: {AB,AC,CD} or {AB,BC,CD}.
AB is uniquely lightest across {A}; CD is uniquely lightest across {D}. AC and BC tie as the cheapest connection between those two components. Choose one, total 1+2+3=6. Taking both AC and BC with CD costs 7.

Source: 07 Minimum Spanning Tree, pp. 2-6; recording M1, 06:42-17:57.

Answers and explanations

27 Prim: cheapest attachment edges

Q79 - D (Easy)
AC, key 2
The initial tree is {A}; only AB and AC cross its boundary. Choose AC=2; globally smaller BC is not yet incident to the tree.

Q80 - D (Medium)
key[D]=5; key[E]=10; next BD
C offers D=8 and E=10 and improves B to 1. Adding B improves D to 5. E cannot use DE until D belongs to the tree.

Q81 - B (Hard)
{AC,CB,BD,DE}; weight 10; parent(E)=D
Edges enter as AC(2), CB(1), BD(5), DE(2). When D joins, E improves from 10 via C to 2 via D. Prim keys are attachment weights, not distances from A.

Source: 07 Minimum Spanning Tree, pp. 8-20; recording M1, 20:55-26:40.

28 Union-find: naive linking and compression

Q82 - A (Easy)
C
Follow A->B->C and stop where parent equals the vertex itself. The representative is the root, not necessarily the smallest label.

Q83 - A (Medium)
(B,C,D,D)
First A points to B. Then root(A)=B points to C. Finally root(A)=C points to D. This creates chain A-B-C-D, showing why naive finds can be linear.

Q84 - B (Hard)
(B,D,D,D,D); partition unchanged
The find path is B-C-D, so B and C point to D afterward; A is not traversed and still points to B. E already points to D. Compression changes shape, not membership.

Source: 07 Minimum Spanning Tree, pp. 22-32; recording M2, 08:57-16:14.

Answers and explanations

29 Union by rank with path compression

Q85 - D (Easy)
p[A]=B; rank[B]=1
The tie rule chooses the second root B. Only its rank increases; A joins B rather than staying a separate root.

Q86 - A (Medium)
p=(B,D,D,D); rank[D]=2
The first two unions produce roots B and D of rank 1. Union(B,D) attaches B below D and raises D to rank 2. A is not found by that union and still points to B.

Q87 - D (Hard)
p=(D,D,D,D); rank[D] remains 2 even though the actual tree height is 1.
Find(A) compresses A-B-D to A-D. Union(A,C) finds the same root D twice and does nothing. Rank is a historical balance bound, not the current compressed height.

Source: 07 Minimum Spanning Tree, pp. 33-35; recording M2, 17:40-24:42.

30 Kruskal: accepted edges and forest state

Q88 - D (Easy)
{AB,BC}
AB(1) joins A/B; BC(2) joins their component to C. Both connect distinct components at their scan times.

Q89 - C (Medium)
Accept AB,BC,CD; reject AC; components {A,B,C,D}, {E}.
AC(3) closes triangle A-B-C and is rejected. CD(4) connects D to the ABC component and is accepted. E is still isolated.

Q90 - C (Hard)
{AB,BC,CD,DE}; weight 13; 6 edges examined
Scan AB accept, BC accept, AC reject, CD accept, BD reject, DE accept. Four accepted edges span five vertices with weight 1+2+4+6=13. CE is never needed.

Source: 07 Minimum Spanning Tree, pp. 37-57; recording M3, 03:24-03:59, 08:08-19:18.

Sources and traceability

All page references in answers are physical PDF page numbers. Animation builds repeat printed slide numbers. Recording IDs below resolve the short references printed beside each answer topic.

01_Sorting.pdf (24 physical pages)

S1 w01_sorting_1_introduction; S2 w01_sorting_2_insertion_sort; S3 w02_sorting_3_merge_sort; S4 w02_sorting_4_quicksort.

02_LinearTimeSorting.pdf (23 physical pages)

S5 w02_sorting_5_counting_sort; S6 w02_sorting_6_radix_sort.

03_Heaps.pdf (48 physical pages)

H1 w03_heaps_1_structure; H2 w03_heaps_2_building; H3 w03_heaps_3_priority_queue; H4 w03_heaps_4_heapsort.

04_IntroToGraphs.pdf (23 physical pages)

G1 w04_graphs_1_introduction.

05_GraphSearch.pdf (44 physical pages)

G2 w04_graphs_2_bfs; G3 w04_graphs_3_dfs; G4 w05_graphs_4_dags.

06_ShortestPath.pdf (29 physical pages)

P1 w06_graphs_5_shortest_paths; P2 w06_graphs_6_dijkstra.

07_MinimumSpanningTree.pdf (57 physical pages)

M1 w07_graphs_7_mst_prim; M2 w07_graphs_8_union_find; M3 w07_graphs_9_kruskal.

General introduction recording

w01_sc2001_introduction covers course organisation and learning advice; it adds no separate algorithm topic here. Together with the 19 topic recordings listed above, the source collection has 20 recordings.

Where the source material was read

Readable local PDFs: /Users/joelloh/Documents/School/SC2001/SC2001_slides_recordings/. These match the seven filenames supplied in the Google Drive course folder.

Timestamped automated transcripts: /Users/joelloh/Documents/Code/WaifuTutor/edit/series/transcripts/*.md. Existing source-review notes and the study-note source ledger were used to locate relevant recording passages and cross-check known caveats.

The cloud recording directory timed out on listing. Existing local transcripts provided recording coverage. Source references identify the taught method; all question inputs, diagrams, distractors and explanations in this bank are newly authored practice examples.

Conventions worth vetting

Quicksort bounds. Sorting pp. 18 and 20 mix end-index conventions. Topic 4 explicitly uses inclusive bounds with the pivot at hi and excludes its final index from recursive calls. Topic 3 separately uses the slide's half-open merge-sort bounds.

Counting positions. Linear Time Sorting pp. 6-8 use cumulative-minus-one last positions, while p. 20 uses cumulative counts. Topic 6 consistently uses p. 20: decrement first, then write into the zero-based output.

Heap and queue rules. Min-heaps are used for construction and the minimum-priority queue. Ascending heapsort explicitly switches to a max-heap and excludes the sorted suffix from repair.

Directed degrees. Intro to Graphs p. 16 repeats "incoming" in the outdegree prose. Outdegree counts outgoing edges. Topic 14 follows the correct definition and uses fresh examples.

DFS classifications. Graph Search p. 34 rank tests apply after tree edges are identified. Topic 18 uses separate preorder/postorder ranks, not a shared timestamp counter. Topic 19 distinguishes directed back edges from reverse scans of undirected parent edges.

Topological order. Graph Search p. 44 says postorder. The DAG recording corrects this to reverse postorder (26:26-28:58) under prerequisite-forward arrows. Topic 21 uses reverse finishing order over a full DFS forest.

Bellman-Ford detection. Shortest Path p. 14 overstates that an improving edge must itself lie on a negative cycle. The correct conclusion is a source-reachable negative cycle; the improving edge may be downstream. In-place passes may propagate multiple edges, depending on scan order.

DAG shortest-path scope. Topic 25 is a tutorial application mentioned in recording P1 around 34:17-34:36, derived from topological ordering and relaxation. The full algorithm is not separately developed in the supplied shortest-path deck.

MST ties and union-find. A tied light cut edge belongs to some MST; a unique light cut edge belongs to every MST. The MST/Prim recording clarifies ties. Union-find tie direction is specified per topic because the slides show different naive link directions; rank does not fall after path compression.

Review focus: correctness of course scope; whether the Easy/Medium/Hard split matches the test; preferred diagram density; and whether the explanations should be hidden in a later student-only version.